

Session 04 - Error Handling, Custom Errors, Inheritance, Overriding, Top Casting

Naveen AutomationLabs | 2026 Playwright Batch G1

1. Error Handling - try/catch/finally

When something goes wrong at runtime (wrong JSON, divide by zero, file not found), JavaScript throws an Error. Use try-catch-finally to handle it gracefully instead of crashing.

```
function parsing() {
  try {
    let result = JSON.parse('{ "name": "Tom" }');
    console.log(result);
  }
  catch (error) {
    console.log(error);
  }
  finally {
    console.log('close the DB connection');
  }
}

parsing();
console.log('Done'); // code continues after try-catch
```

- try: code that might throw an error.
- catch: runs only if an error occurs inside try.
- finally: runs no matter what -- error or no error. Good for cleanup like closing DB connections.

2. throw - Throwing Errors Manually

You can throw your own errors using the 'throw' keyword. This is useful for validating inputs.

```
function div(a: number, b: number): number {
  if (b === 0) {
    throw new Error("can not be divided by zero");
  }
  return a / b;
}

let r1 = div(10, 10);
console.log(r1); // 1

// div(10, 0); // throws Error: can not be divided by zero
```

3. Custom Error Classes

You can create your own error classes by extending the built-in Error class. This helps identify what kind of error occurred.

```

class ElementError extends Error {
  constructor(message: string) {
    super(message);
  }
}

class BrowserError extends Error {
  constructor(message: string) {
    super(message);
  }
}

class FrameworkError extends Error {
  constructor(message: string) {
    super(message);
  }
}

```

Using custom errors:

```

let browser = 'naveen';
switch (browser) {
  case 'chrome':
    console.log('open chrome');
    break;
  default:
    console.log('plz pass the right browser name....');
    throw new BrowserError('==INVALID BROWSER== ' + browser);
}

// The line below will NOT execute because the error was thrown:
console.log('entering the app url');

```

Custom error classes make your framework logs clearer. Instead of a generic Error, you see BrowserError or ElementError -- much easier to debug.

4. The 'never' Return Type

A function with return type 'never' will never finish normally. It always throws an error or runs an infinite loop.

```

function m1(): never {
  throw new Error('some error');
}

m1(); // always throws, never returns

```

5. Classes and Inheritance

TypeScript supports classes with inheritance using the 'extends' keyword. A child class inherits all properties and methods from the parent class.

```

class Car {
  start() {
    console.log('Car -- start');
  }
  stop() {
    console.log('Car -- stop');
  }
  refuel() {
    console.log('Car -- refuel');
  }
}

```

```

}
}

class BMW extends Car {
  override start() {
    console.log('BMW -- start');
  }
  autoParking() {
    console.log('BMW -- auto parking');
  }
}

let bmw: BMW = new BMW();
bmw.start(); // BMW -- start (overridden)
bmw.stop(); // Car -- stop (inherited)
bmw.refuel(); // Car -- refuel (inherited)
bmw.autoParking(); // BMW -- auto parking (own method)

```

6. Method Overriding

When a child class defines a method with the same name as the parent, it overrides the parent method. Use the 'override' keyword to make it explicit.

```

class BMW extends Car {
  override start() {
    console.log('BMW -- start');
  }
}

// When you call bmw.start(), the child class version runs.

```

7. Top Casting (Upcasting)

A child class object can be referred to by a parent class reference variable. This is called upcasting or top casting. The IS-A relationship: BMW IS-A Car.

```

// Top casting: child object with parent reference
let c1: Car = new BMW();
c1.start(); // BMW -- start (overridden method runs)
c1.stop(); // Car -- stop
c1.refuel(); // Car -- refuel
// c1.autoParking(); // Error: not available on Car type

// Down casting: parent object with child reference
// let b1: BMW = new Car(); // NOT ALLOWED -- will not compile

```

With top casting, you can only access methods defined in the parent type. Child-only methods like autoParking() are not visible through the parent reference.